Date: 08-11-2026

Lab # 08 TO UNDERSTAND BIG-O NOTATION

Lab Objective:

- Understand the concept of Big-O notation.
- Analyze the time complexity of algorithms.
- Implement simple algorithms and evaluate their time complexity.

Theory:

Introduction

Big-O notation is a mathematical concept used in computer science to describe the performance or complexity of an algorithm. It helps evaluate how an algorithm's runtime or space requirements grow as the size of the input increases.

Why Learn Big-O Notation?

1. **Efficiency Matters**: With large data, a poorly designed algorithm can lead to significant delays.
2. **Comparing Algorithms**: It provides a standard way to compare different algorithms based on their scalability.
3. **Optimization**: Helps in identifying and improving inefficient parts of code.

Core Concepts

1. What is Big-O Notation?

- Big-O notation describes the **worst-case scenario** for the runtime of an algorithm.
- It measures the **upper bound** of time or space as the input size (nnn) grows.

For example:

- **Algorithm A** has a complexity of O(n)O(n)O(n), and **Algorithm B** has O(n2)O(n^2)O(n2).
- If n=1000n = 1000n=1000, Algorithm A would take 1000 steps, while Algorithm B would take 1,000,000 steps.

1. **Common Big-O Notations**:
    - O(1) Constant time.
    - O(logn): Logarithmic time.
    - O(n): Linear time. o O(n^2): Quadratic time.

Date: 08-11-2026

Part 2: Practical Tasks

Task 1: Analyze Time Complexity

For each of the following snippets, determine the time complexity.

1. **Snippet 1: Constant Time**

```
def constant_time_example(n): return
    n * 2
```

2. **Snippet 2: Linear Time**

```
def linear_time_example(arr):
    for element in arr:
        print(element)
```

3. **Snippet 3: Quadratic Time**

```
def quadratic_time_example(arr):
    for i in arr:
        for j in arr: print(i,
            j)
```

Task 2: Compare Algorithms

1. Implement two algorithms for finding the maximum element in a list.
    - **Algorithm 1: Linear Search**

```
def
    find_max_linear(arr)
    : max_value = arr[0]
    for num in arr:
        if num > max_value:
            max_value = num return
    max_value
```

    - **Algorithm 2: Built-in Function**

```
def find_max_builtin(arr): return
    max(arr)
```

2. Test both algorithms with input sizes of varying lengths (e.g., 10, 100, 1000).
3. Record the execution time using `time` module.

Date: 08-11-2026

Example:

```
      import time
 arr = list(range(1, 1001)) # Adjust size
      start = time.time()
      find_max_linear(arr)
      end = time.time()
      print("Execution Time (Linear):", end - start)

      start = time.time()
      find_max_builtin(arr)
      end = time.time()
      print("Execution Time (Built-in):", end - start)
```

Task 3: Identify Growth Rates

Write a Python function that prints the time taken to calculate the sum of the first n integers for different values of n: 10, 100, 1000, 10000.

```
def sum_of_integers(n): total
    = 0 for i in range(1, n +
    1):
        total += i
    return total import

time

for n in [10, 100, 1000, 10000]:
    start = time.time()
    sum_of_integers(n)
    end = time.time()
    print(f"Time for n={n}: {end - start}")
```

Exercise:

1. What does Big-O notation represent in algorithm analysis?
2. List three advantages of analyzing an algorithm's time complexity.
3. Match each algorithm to its Big-O complexity:

   (a) Binary Search

   (b) Bubble Sort (worst case)

   (c) Accessing an element in an array

   (d) Merge Sort

Date: 08-11-2026

Introduction:

Big-O notation is a mathematical concept used in computer science to describe the performance and complexity of an algorithm. It helps us understand how the execution time or memory requirements of an algorithm increase when the input size becomes larger. Big-O notation is useful for comparing different algorithms and identifying which algorithm is more efficient. Common types of Big-O notation include O(1), O(log n), O(n), and $O(n^2)$.

Abstract:

In this lab, we studied the concept of Big-O notation and its importance in algorithm analysis. We learned how to calculate the time complexity of different algorithms, including constant time, linear time, and quadratic time. We also compared two algorithms for finding the maximum element in a list and measured their execution time using Python's time module. Finally, we observed how execution time changes with different input sizes.

Procedure:

1. First, we studied the concept of Big-O notation and its importance in data structures.
2. We learned about different time complexities, including O(1), O(log n), O(n), and $O(n^2)$.
3. We analyzed three Python code snippets to identify their time complexity.
4. We implemented two algorithms to find the maximum element in a list using linear search and the built-in max() function.
5. We used Python's time module to calculate and compare the execution time of both algorithms.
6. We implemented a function to calculate the sum of the first n integers.
7. We tested the function with different input sizes: 10, 100, 1000, and 10000.
8. Finally, we observed the execution time and understood how algorithm performance changes with input size.

Date: 08-11-2026

Q1. What does Big-O notation represent in algorithm analysis?
Answer:
Big-O notation represents the time or space complexity of an algorithm. It describes how the performance of an algorithm changes as the input size increases. It is commonly used to represent the worst-case time complexity of an algorithm.

Q2. List three advantages of analyzing an algorithm's time complexity.
Answer:

1. **Efficiency:** It helps us identify how efficiently an algorithm works.
2. **Comparison:** It allows us to compare different algorithms and select the better one.
3. **Optimization:** It helps identify slow parts of a program and improve its performance.

Q3. Match each algorithm to its Big-O complexity.
(a) Binary Search

(b) Bubble Sort (worst case)

(c) Accessing an element in an array

(d) Merge Sort

(a) Binary Search – O(log n)

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = [10, 20, 30, 40, 50, 60]
print("Element found at index:", binary_search(arr, 40))
```

```
Element found at index: 3
```

Date: 08-11-2026

(b) Bubble Sort (Worst Case) – O(n²)

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


arr = [50, 40, 30, 20, 10]
print("Sorted Array:", bubble_sort(arr))
```

```
Sorted Array: [10, 20, 30, 40, 50]
```

(c) Accessing an Element in an Array – O(1)

```
arr = [10, 20, 30, 40, 50]

index = 2

print("Element at index", index, "is:", arr[index])
```

```
Element at index 2 is: 30
```

(d) Merge Sort: O(n log n)

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
```

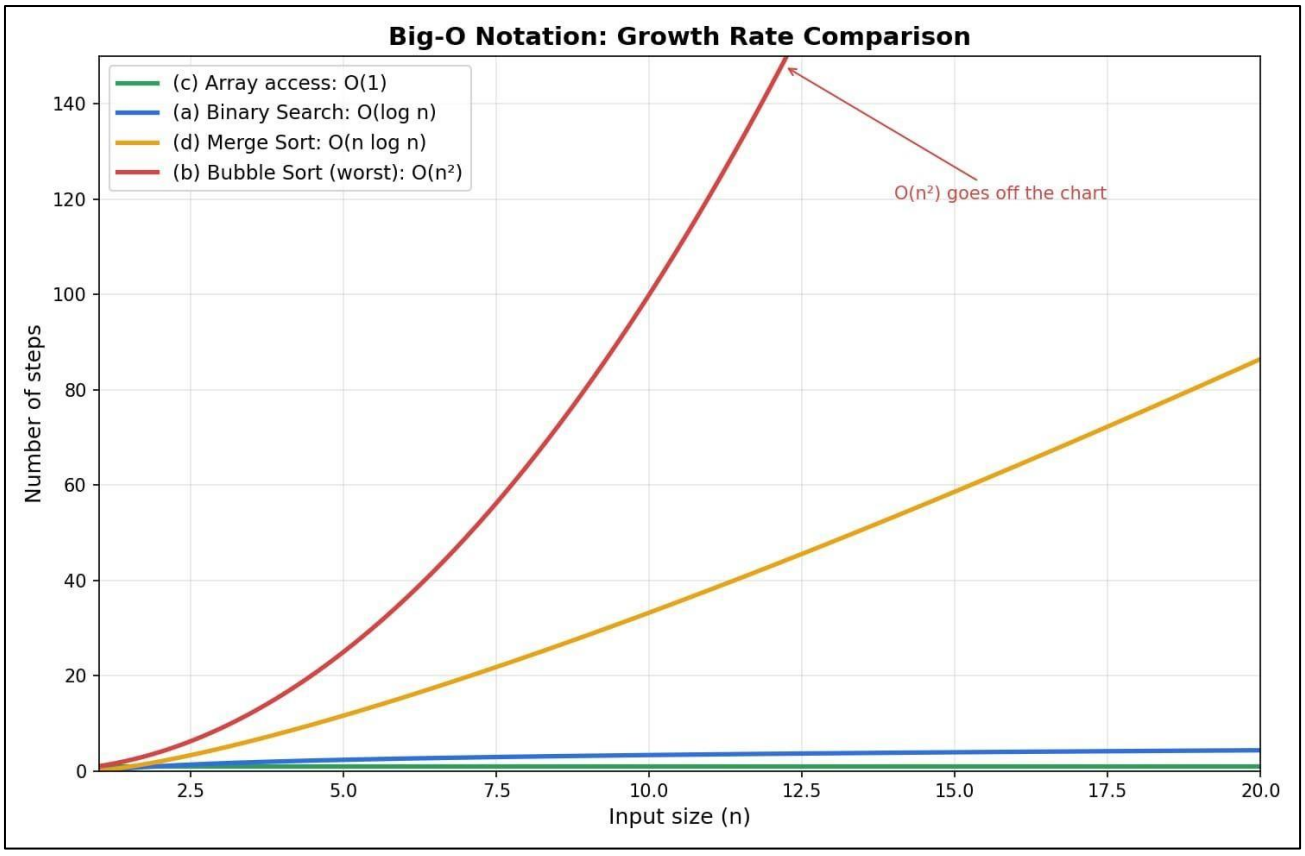
Date: 08-11-2026

```
            while j < len(right):
                arr[k] = right[j]
                j += 1
                k += 1

        return arr


    arr = [50, 20, 40, 10, 30]
    print("Sorted Array:", merge_sort(arr))
```

```
Sorted Array: [10, 20, 30, 40, 50]
```



Date: 08-11-2026

Conclusion

In this lab, we successfully understood the concept of Big-O notation and its importance in algorithm analysis. We learned how to identify the time complexity of different algorithms, including constant, logarithmic, linear, and quadratic time. We also implemented algorithms to find the maximum element in a list and measured their execution time using Python. This lab helped us understand how input size affects algorithm performance and how Big-O notation helps us choose efficient algorithms.